

Módulo 04 — Respostas Conceituais

Trilha de Docker | UniSENAI 2026

Exercício 2 — Explicando as instruções e a importância do Dockerfile

FROM — qual é sua função?

FROM é a **primeira instrução obrigatória de todo Dockerfile**. Ela define a imagem base da qual a nova imagem será construída — ou seja, o ponto de partida.

Todo ambiente precisa de um sistema operacional mínimo ou de um runtime já configurado. Em vez de montar tudo do zero, o Docker parte de uma imagem existente. No exemplo do módulo:

```
FROM alpine
```

Isso diz: "comece com o Linux Alpine". A partir daí, todas as outras instruções (**COPY**, **RUN**, **CMD**, etc.) são executadas sobre esse sistema base.

Outros exemplos comuns de uso:

- **FROM node:18** → já vem com Node.js instalado, pronto para aplicações JavaScript
- **FROM python:3.11-slim** → Python instalado, imagem enxuta
- **FROM ubuntu:22.04** → Ubuntu completo como base

CMD — qual é sua função?

CMD define o **comando padrão que será executado quando o contêiner iniciar**. Ele só é executado em tempo de execução (quando alguém faz **docker run**), não durante o build da imagem.

```
CMD ["echo", "Bem-vindo ao Docker!"]
```

Pontos importantes sobre o **CMD**:

- Só pode haver **um CMD por Dockerfile** — se houver mais de um, apenas o último vale

Pode ser **sobrescrito** passando um comando ao final do `docker run`:

`docker run hello-docker echo "Outro comando"#` Ignora o CMD do Dockerfile e executa "echo Outro comando"

-
- Existe em duas formas:
 - **Exec form** (recomendada): `CMD ["echo", "texto"]` — executa diretamente, sem shell
 - **Shell form**: `CMD echo texto` — executa via `/bin/sh -c`, com shell intermediário

Por que o Dockerfile é importante para padronização de ambientes?

Antes do Docker, o problema clássico de qualquer equipe era: *"funciona na minha máquina"*. Cada desenvolvedor tinha versões diferentes de linguagens, bibliotecas e configurações instaladas no próprio computador. O servidor de produção era diferente do servidor de homologação, que era diferente da máquina de desenvolvimento.

O Dockerfile resolve isso ao ser um **documento executável do ambiente**. Em vez de uma lista de instruções manuais ("instale o Node 18, depois rode npm install, depois configure isso..."), o Dockerfile é um script que o Docker executa de forma idêntica em qualquer máquina.

Isso traz três benefícios diretos:

- 1. Reprodutibilidade total** — qualquer pessoa que clone o projeto e rode `docker build` vai ter exatamente o mesmo ambiente, independente do sistema operacional do computador dela.
- 2. Versionamento do ambiente junto com o código** — o Dockerfile fica no repositório Git junto com o código-fonte. Quando o projeto muda de Node 16 para Node 18, isso fica registrado no histórico do Git como qualquer outra alteração de código.
- 3. Automação de deploys** — pipelines de CI/CD (GitHub Actions, GitLab CI, Jenkins) conseguem construir a imagem automaticamente a partir do Dockerfile a cada commit, garantindo que o ambiente de produção seja sempre atualizado de forma controlada e rastreável.

-